

Estructura de datos

Recursión

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2026-II

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO

- 1 Concepto de recursión
- 2 Funcionamiento de la recursión en memoria
- 3 Diseño de algoritmos recursivos
- 4 Algoritmos recursivos fundamentales



Recursión

Definición

La **recursión** es una **técnica** para **resolver problemas** en la que un problema se **expresa** en términos de **instancias más pequeñas del mismo problema**.

- Permite **dividir un problema** en problemas más pequeños.
- **Cada llamada** trabaja con una instancia diferente del problema.
- El proceso continúa hasta **alcanzar una condición de terminación**.
- En programación, normalmente **se implementa** mediante un método que **se invoca a sí mismo**.

¡Importante!

Una **solución recursiva** debe **tener** una forma de **detener las llamadas**.

Definición recursiva de un problema

Un **problema** puede definirse **recursivamente** cuando su **solución** puede expresarse utilizando una **versión más pequeña** del mismo problema.

Ejemplo: factorial

El **factorial** de un **número natural** n se define como:

$$n! = \begin{cases} 1, & n = 0 \\ n(n-1)!, & n > 0 \end{cases}$$

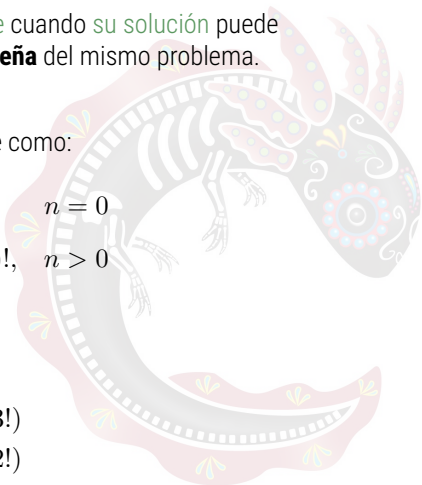
Por ejemplo:

$$4! = 4(3!)$$

$$3! = 3(2!)$$

$$2! = 2(1!)$$

$$1! = 1(0!)$$



Componentes de una solución recursiva

Una **solución recursiva** contiene al menos dos elementos:

■ Caso base

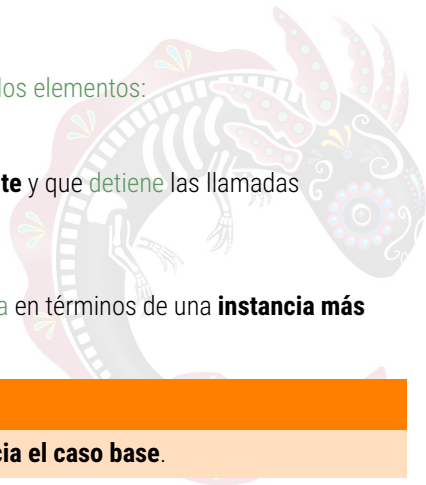
Condición que puede resolverse **directamente** y que **detiene** las llamadas recursivas.

↻ Caso recursivo

Parte de la solución que **expresa el problema** en términos de una **instancia más pequeña del mismo problema**.

¡Importante!

Cada llamada debe realizar un **progreso hacia el caso base**.



Caso base

Definición

El **caso base** representa una **instancia del problema** cuya solución se conoce directamente y **no requiere realizar otra llamada recursiva**.

Para el factorial:

$$0! = 1$$

En **Java** ☕:

```
if (n == 0) {  
    return 1;  
}
```

¡Importante!

Si una función recursiva **no posee un caso base alcanzable**, las llamadas continuarán **indefinidamente** hasta **agotar la pila**.

Caso recursivo

Definición

El **caso recursivo** transforma el problema actual en una **instancia más pequeña del mismo problema**.

Para el factorial:

$$n! = n(n - 1)!$$

En **Java** ☞:

```
return n * factorial(n - 1);
```

La llamada:

```
factorial(4)
```

genera una **nueva llamada**:

```
factorial(3)
```

Implementación recursiva del factorial

```
public static long factorial(int n) {  
    // Caso base  
    if (n == 0) {  
        return 1;  
    }  
  
    // Caso recursivo  
    return n * factorial(n - 1);  
}
```

Estructura

$\underbrace{\text{if (n == 0)}}_{\text{caso base}}$

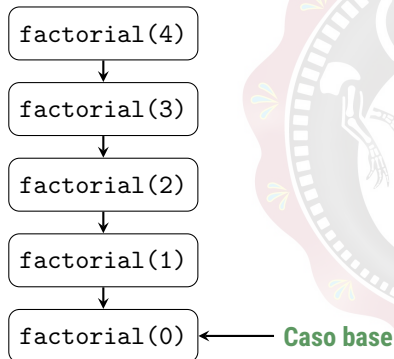
$\underbrace{\text{factorial(n - 1)}}_{\text{caso recursivo}}$

¡Importante!

El argumento $n - 1$ **garantiza** que **cada llamada** se aproxima al **caso base**.

Progreso hacia el caso base

Una **llamada recursiva** debe **modificar el problema** de forma que **se aproxime al caso base**.



Expansión y retorno

La ejecución de un **algoritmo recursivo** puede observarse en **dos etapas**:

Llamadas

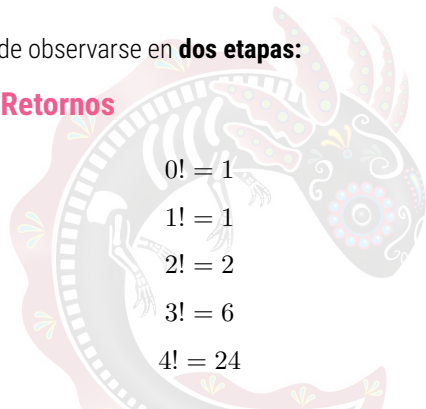
$4!$
 $4(3!)$
 $4(3(2!))$
 $4(3(2(1!)))$
 $4(3(2(1(0!))))$

Retornos

$0! = 1$
 $1! = 1$
 $2! = 2$
 $3! = 6$
 $4! = 24$

¡Importante!

Al alcanzar el **caso base**, las **llamadas pendientes** comienzan a **regresar sus resultados en orden inverso**.



Recursión indirecta

Definición

Existe **recursión indirecta** cuando un método llama a otro método que, directa o indirectamente, **vuelve a llamar al primero**.

```
static void metodoA(int n) {  
    if (n > 0) {  
        metodoB(n - 1);  
    }  
}  
  
static void metodoB(int n) {  
    if (n > 0) {  
        metodoA(n - 1);  
    }  
}
```

metodoA() → metodoB() → metodoA() → ...

Llamadas a métodos

Cada vez que un **método** es **invocado**, el programa necesita **almacenar información** asociada con esa **llamada particular**.

```
public static long factorial(int n) {  
    if (n == 0) {  
        return 1;  
    }  
    return n * factorial(n - 1);  
}
```

Al ejecutar:

```
factorial(3);
```

se producen **las llamadas**:

$\text{factorial}(3) \rightarrow \text{factorial}(2) \rightarrow \text{factorial}(1) \rightarrow \text{factorial}(0)$

¿Cómo recuerda el programa el valor de **n** **correspondiente a cada llamada?**

Registros de activación

Definición

Cada llamada a un método **genera** un **registro de activación (stack frame)** que **contiene** la **información necesaria** para administrar dicha llamada.

Entre otros elementos, puede contener:

- **Parámetros** recibidos por el **método**.
- **Variables locales**.
- **Información necesaria** para **regresar** al método que **realizó la llamada**.
- **Valores temporales** necesarios durante la **ejecución**.

¡Importante!

Cada **llamada recursiva** posee su **propio registro de activación**.

Pila de llamadas

Los **registros de activación** se organizan mediante una **pila de llamadas (call stack)**.



<code>factorial(0)</code>	$n = 0$
<code>factorial(1)</code>	$n = 1$
<code>factorial(2)</code>	$n = 2$
<code>factorial(3)</code>	$n = 3$

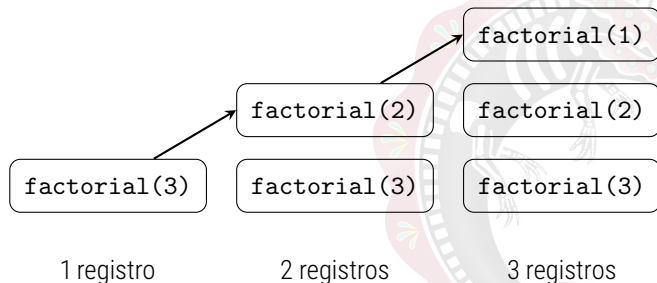
Tope

LIFO

La última llamada realizada es la **primera que debe finalizar**.

Crecimiento de la pila

Para ejecutar `factorial(3)`, cada llamada agrega un **nuevo registro** a la pila:



¡Importante!

La **recursión** requiere **memoria adicional** para mantener las **llamadas pendientes**.

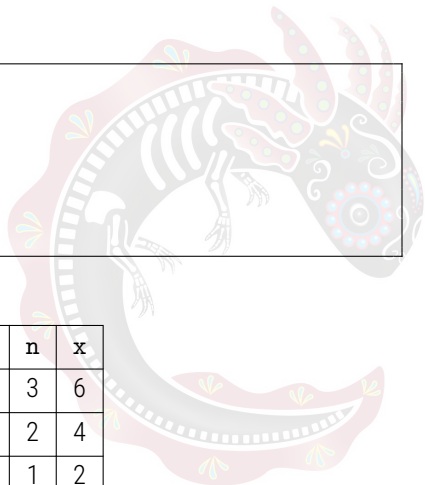
Variables locales y parámetros

Considere el **siguiente método**:

```
static int ejemplo(int n) {  
    int x = n * 2;  
    if (n == 0)  
        return x;  
    return x + ejemplo(n - 1);  
}
```

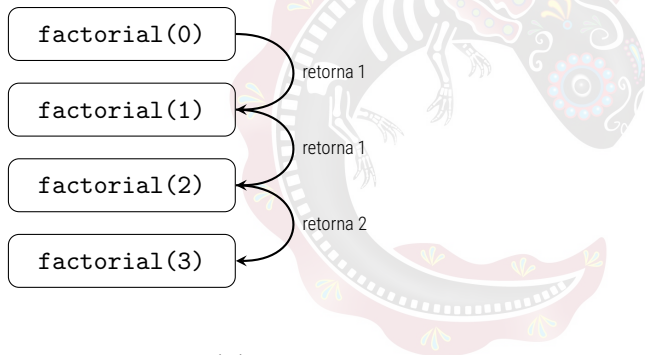
Cada **llamada** posee sus **propias variables**:

Llamada	n	x
ejemplo(3)	3	6
ejemplo(2)	2	4
ejemplo(1)	1	2
ejemplo(0)	0	0



Retorno de las llamadas

Una vez alcanzado el **caso base**, los registros de activación **comienzan a retirarse** de la pila en **orden inverso**.



Finalmente:

$$\text{factorial}(3) = 6$$

Desbordamiento de pila

¿Qué ocurre si **nunca** se **alcanza** el **caso base**?

```
static void infinito(int n) {  
    System.out.println(n);  
    infinito(n + 1);  
}
```

Las llamadas **continúan acumulándose**:

`infinito(0) → infinito(1) → infinito(2) → ...`

¡Importante!

Cuando la **pila de llamadas** ya **no dispone** de **espacio suficiente**, **Java** ☞ produce un error **StackOverflowError**.

Diseño de algoritmos recursivos

Definición

Diseñar un **algoritmo recursivo** consiste en **expresar la solución** de un problema utilizando la **solución de una instancia más pequeña** del mismo problema.

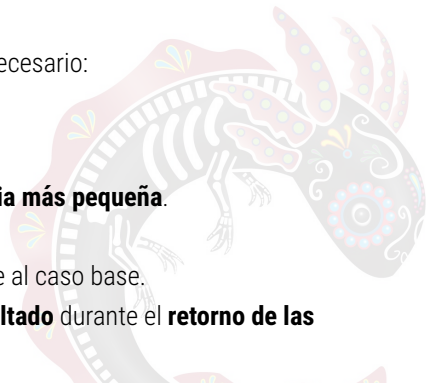
Una **estrategia general** consiste en **identificar**:

- 1 ¿Cuál es la **instancia más sencilla** del problema?
- 2 ¿Cómo puede **reducirse el problema** a una instancia más pequeña?
- 3 ¿Qué **información** debe **procesarse** en la llamada actual?
- 4 ¿Cómo se **combina** el **resultado actual** con el **resultado recursivo**?
- 5 ¿La reducción **garantiza alcanzar** el **caso base**?

Estrategia de diseño

Para **construir** una **solución recursiva**, es necesario:

- 1 **Definir el problema** y sus **parámetros**.
- 2 **Identificar** uno o más **casos base**.
- 3 **Determinar** cómo **obtener** una **instancia más pequeña**.
- 4 **Formular** el **caso recursivo**.
- 5 Verificar que cada llamada se aproxime al caso base.
- 6 Comprobar cómo se **construye el resultado** durante el **retorno de las llamadas**.



Pregunta fundamental

Si conozco la **solución** de una **instancia** más pequeña, **¿cómo puedo utilizarla** para **resolver** la **instancia actual**?

Descomposición de problemas

Considere el **problema** de calcular:

$$S(n) = 1 + 2 + 3 + \cdots + n$$

Podemos **separar** el último elemento:

$$S(n) = n + \underbrace{(1 + 2 + \cdots + (n - 1))}_{S(n-1)}$$

Por tanto:

$$S(n) = \begin{cases} 0, & n = 0 \\ n + S(n - 1), & n > 0 \end{cases}$$

Descomposición

El problema $S(n)$ se transforma en una **instancia más pequeña**:

$$S(n - 1)$$

De la definición al algoritmo

La **definición matemática**:

$$S(n) = \begin{cases} 0, & n = 0 \\ n + S(n - 1), & n > 0 \end{cases}$$

puede **traducirse** directamente a **Java** ☞:

```
public static int suma(int n) {
    if (n == 0) {
        return 0;
    }
    return n + suma(n - 1);
}
```

Correspondencia

Caso base	→	condición de terminación
Caso recursivo	→	llamada con un problema menor

Seguimiento de llamadas recursivas

Para **comprender** un **algoritmo recursivo** es útil realizar una **traza de ejecución**.

Para **calcular** $S(4)$:

$$\begin{aligned} S(4) &= 4 + S(3) \\ &= 4 + 3 + S(2) \\ &= 4 + 3 + 2 + S(1) \\ &= 4 + 3 + 2 + 1 + S(0) \\ &= 4 + 3 + 2 + 1 + 0 \end{aligned}$$

Después **comienza** el **retorno**:

$$0 \rightarrow 1 \rightarrow 3 \rightarrow 6 \rightarrow 10$$

Por lo tanto:

$$S(4) = 10$$

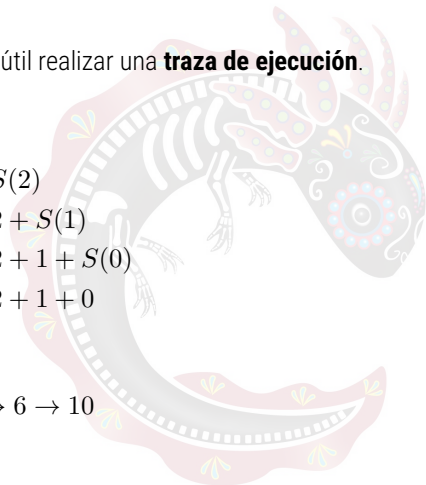
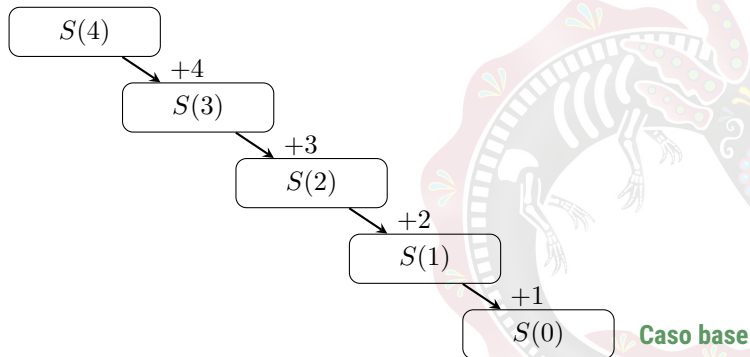


Diagrama de llamadas



Utilidad

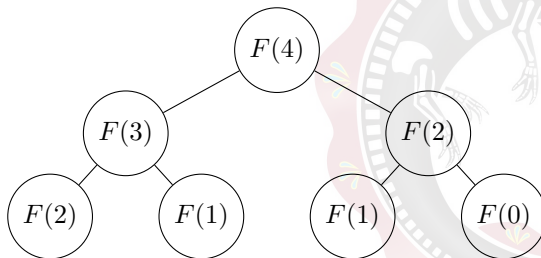
Los diagramas permiten **visualizar** la **secuencia de llamadas** y cómo **disminuye** el **tamaño del problema**.

Árboles de recursión

Algunos algoritmos generan **más de una llamada recursiva** en cada ejecución.

$$F(n) = F(n - 1) + F(n - 2)$$

Por ejemplo:



¡Importante!

Un **árbol de recursión** permite **identificar llamadas repetidas** y **visualizar el crecimiento** del número de operaciones.

Recursión sobre números

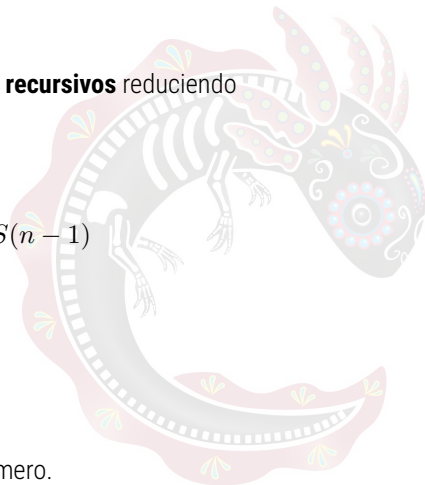
Los números permiten **construir problemas recursivos** reduciendo progresivamente su valor.

Suma de los primeros n números

$$S(n) = n + S(n-1)$$

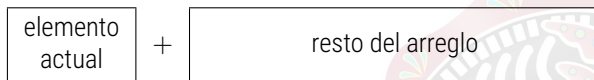
Otros ejemplos:

- Factorial.
- Potencias.
- Máximo común divisor.
- Procesamiento de los dígitos de un número.



Recursión sobre arreglos

Un **arreglo** puede **procesarse** considerando:



Por ejemplo, para **sumar sus elementos**:

```
public static int suma(int[] data, int index) {  
    if (index == data.length) {  
        return 0;  
    }  
    return data[index] + suma(data, index + 1);  
}
```

Para:

[10, 20, 30, 40]

se genera:

$10 + (20 + (30 + (40 + (0))))$

Recursión sobre cadenas

Una cadena puede procesarse carácter por carácter mediante un índice.

```
public static void mostrar(String text, int index) {  
    if (index == text.length()) {  
        return;  
    }  
    System.out.println(text.charAt(index));  
    mostrar(text, index + 1);  
}
```

Para:

"JAVA"

las llamadas procesan:

$J \rightarrow A \rightarrow V \rightarrow A$

Idea

El índice permite representar la parte de la cadena que **todavía falta procesar**.

Factorial

$$n! = \begin{cases} 1, & n = 0 \\ n(n-1)!, & n > 0 \end{cases}$$

```
public static long factorial(int n) {  
    if (n == 0) {  
        return 1;  
    }  
    return n * factorial(n - 1);  
}
```

Para $n = 4$:

$$\begin{aligned} 4! &= 4(3!) \\ &= 4(3(2!)) \\ &= 4(3(2(1!))) \\ &= 4(3(2(1(0!)))) \\ &= 24 \end{aligned}$$

Potencia

Para un **exponente entero no negativo**:

$$a^n = \begin{cases} 1, & n = 0 \\ a \cdot a^{n-1}, & n > 0 \end{cases}$$

```
public static long potencia(int base, int exp) {
    if (exp == 0) {
        return 1;
    }
    return base * potencia(base, exp - 1);
}
```

Por ejemplo:

$$\begin{aligned} 2^4 &= 2(2^3) \\ &= 2(2(2^2)) \\ &= 2(2(2(2^1))) \\ &= 2(2(2(2(2^0)))) \\ &= 16 \end{aligned}$$

Máximo común divisor

Definición

El **máximo común divisor** de dos enteros a y b es el **mayor entero positivo** que divide exactamente **a ambos**.

Por ejemplo:

$$\text{mcd}(48, 18) = 6$$

El **algoritmo de Euclides** utiliza la propiedad:

$$\text{mcd}(a, b) = \text{mcd}(b, a \bmod b)$$

hasta **alcanzar**:

$$b = 0$$

Entonces:

$$\text{mcd}(a, 0) = a$$

Algoritmo de Euclides

```
public static int mcd(int a, int b) {  
    if (b == 0) {  
        return a;  
    }  
    return mcd(b, a % b);  
}
```

Para calcular:

$\text{mcd}(48, 18)$

$$\begin{aligned}\text{mcd}(48, 18) &= \text{mcd}(18, 12) \\ &= \text{mcd}(12, 6) \\ &= \text{mcd}(6, 0) \\ &= 6\end{aligned}$$

Suma de elementos de un arreglo

Sea A un arreglo de n elementos:

$$S(A, i) = \begin{cases} 0, & i = |A| \\ A[i] + S(A, i + 1), & i < |A| \end{cases}$$

```
public static int suma(int[] data, int i) {  
    if (i == data.length) {  
        return 0;  
    }  
    return data[i] + suma(data, i + 1);  
}
```

Para $A = [10, 20, 30, 40]$

$$\begin{aligned} S(A, 0) &= 10 + S(A, 1) \\ &= 10 + 20 + S(A, 2) \\ &= 10 + 20 + 30 + S(A, 3) \\ &= 10 + 20 + 30 + 40 + S(A, 4) \\ &= 100 \end{aligned}$$

Búsqueda binaria recursiva

La **búsqueda binaria** puede expresarse **naturalmente** mediante **recursión**.

[5, 12, 18, 25, 31, 42, 57, 63, 78]

En cada llamada:

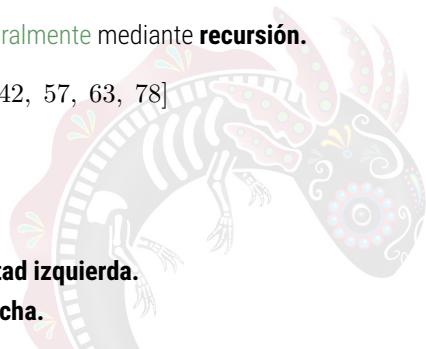
- 1 Se obtiene el **elemento central**.
- 2 Si **coincide**, **termina** la búsqueda.
- 3 Si el valor es **menor**, se **busca** en la **mitad izquierda**.
- 4 Si es **mayor**, se **busca** en la **mitad derecha**.

Caso base

Si el **límite izquierdo** supera al **derecho**:

$$left > right$$

entonces **el elemento no se encuentra**.



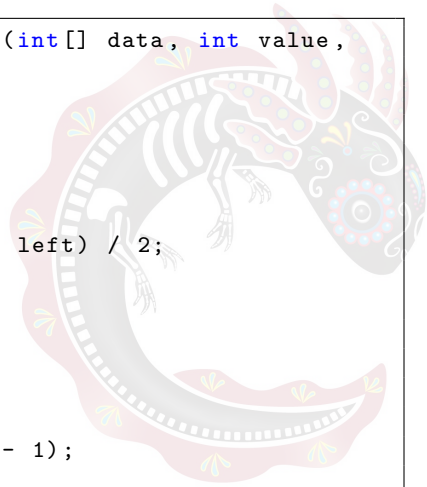
Búsqueda binaria recursiva

```
public static int binarySearch(int[] data, int value,
    int left, int right) {

    if (left > right) {
        return -1;
    }

    int middle = left + (right - left) / 2;

    if (data[middle] == value) {
        return middle;
    }
    if (value < data[middle]) {
        return binarySearch(
            data, value, left, middle - 1);
    }
    return binarySearch(data, value, middle + 1, right);
}
```



Torres de Hanói

Definición

Se tienen **tres torres** y n discos de **diferentes tamaños**. El objetivo es **trasladar todos los discos** desde la torre de **origen** hasta la torre de **destino**.

Reglas

- 1 Mover **un solo disco a la vez**.
- 2 Solo puede **moverse el disco superior**.
- 3 Un **disco grande nunca** puede colocarse sobre **uno pequeño**.

Estrategia recursiva

- 1 Mover $n - 1$ discos al **auxiliar**.
- 2 Mover el disco n al **destino**.
- 3 Mover los $n - 1$ discos **del auxiliar al destino**.

$$H(n) = 2H(n - 1) + 1$$

Torres de Hanói

```
public static void hanoi(int n, char origen, char
    auxiliar, char destino) {

    if (n == 1) {
        System.out.println(origen + " -> " + destino);
        return;
    }

    hanoi(n - 1, origen, destino, auxiliar);
    System.out.println(origen + " -> " + destino);
    hanoi(n - 1, auxiliar, origen, destino);
}
```

Número mínimo de movimientos

Para n discos:

$$2^n - 1$$

Torres de Hanói: ejemplo

Para tres discos:

$$H(3) = 2^3 - 1 = 7$$

Movimiento	Acción
1	$A \rightarrow C$
2	$A \rightarrow B$
3	$C \rightarrow B$
4	$A \rightarrow C$
5	$B \rightarrow A$
6	$B \rightarrow C$
7	$A \rightarrow C$

¡Importante!

El **problema** de n discos se **resuelve** utilizando **dos veces** la **solución** del problema de $n - 1$ discos.

Fin de la presentación

¿Preguntas o comentarios?

